

Introduction to Programming in C#

SWEN1 | BIF3

Table of Contents

- .Net Framework Overview
- The basics of programming in C#
- Further reading



.Net Framework Overview

.Net Platform & Standard

C# is a .NET language which exists in the following implementations

- .NET 5.0 and above (formerly named „.NET Core)
- .NET Framework (until V4.8.2; first presentation in 2000)
- Mono (starting open-source in 2001)

.NET Standard is the specification common to all.

- Set of APIs implemented by each BCL (Base Class Library)
- Enables portability (write once run anywhere; WORA)
- Also a target framework

.Net - A Unified Platform



.Net Platform (formerly named .Net Core)

- .NET is a free, cross-platform, open source developer platform for building many kinds of applications (general purpose).
- .NET is built on a high-performance runtime that is used in production by many high-scale apps in modular manner (NuGet packages)

Cloud apps	Cross-platform client apps	Windows apps	Other app types
Cloud native apps	Desktop apps	Windows Desktop apps	Machine learning
Console apps	Games	- (WinForms), WPF	Internet of Things (IoT)
Serverless functions in the cloud	Mobile apps	- Universal Windows Plattform (UWP)	
Web apps, web APIs, and microservices		Windows Services	

.Net Objectives

- OOP
- Easy deployment (no dependency hell)
- Safe execution
- Performant
- Good developer experience (Visual Studio, Visual Studio Code)
- Provide communication industry standards OOTB (=out of the box)
- Interop-services (legacy)

.Net Components Relationship

- CLR (common language runtime) for management:
  Memory (Garbage Collector)
  - Threads
  - Remoting
  - Enforce type safety (common type system in IL)
  - Compilation
- Byte-Code with JIT-compilation
- Base or framework class lib

.Net Platform Features

.NET features allow developers to productively write reliable and performant code.

- Asynchronous code
- Attributes
- Reflection
- Code analyzers
- Delegates and lambdas
- Events
- Exceptions
- Garbage collection
- Generic types
- LINQ (Language Integrated Query).
- Parallel programming
- Type inference - C#, F#, Visual Basic.
- Type system
- Unsafe code

Further reading

- Introduction to .Net: https://learn.microsoft.com/en-us/dotnet/core/introduction
- .NET | Free. Cross-platform. Open Source: https://dotnet.microsoft.com/en-us/
- Overview of .NET Framework: https://learn.microsoft.com/en-us/dotnet/framework/get-started/overview
- Get started with .NET Framework: https://learn.microsoft.com/en-us/dotnet/framework/get-started/



The Basics of programming in C#

The C# language

- C# (pronounced "See Sharp") is a modern, object-oriented, and type-safe programming language.
- C# enables developers to build many types of secure and robust applications that run in .NET.
- C# has its roots in the C family of languages and will be immediately familiar to C, C++, Java, and JavaScript programmers.
- The C# Language Standard: ECMA-334 (Version 5 – 2017), ISO/IEC 23270:2018
- Common Language Infrastructure: ECMA-335 (Version 6 – 2021), ISO/IEC 23271:2021

C# language features

- Type-safe (strongly typed)
- Object-oriented
- Case-sensitive
- Managed .NET language (implements CLS – common language spec .NET)
- Syntax (e.g.: curly braces) similar to other languages (easy to learn)
- Simplifies complexity (in comparison to C++)
- Feature-rich: Nullable types, Exception handling, Lambda expressions, LINQ (Language Integrated Query), Asynchronous operations (async-await), unified type system, supports reference types and value types, XmlDoc,…

C# Basic Sample

Classical approach:

```
// Program.cs
namespace ConsoleApp
{
    internal class Program
    {
        static void Main(String[] args)
        {
            Console.WriteLine("Hello World!");
        }
    }
}
```

- `namespace` (no mapping to dirs.)
- `using` (hidden here)
- `class`
- method (static, parameters, return)

Using Top-level Statements:

(since .net 7.0)

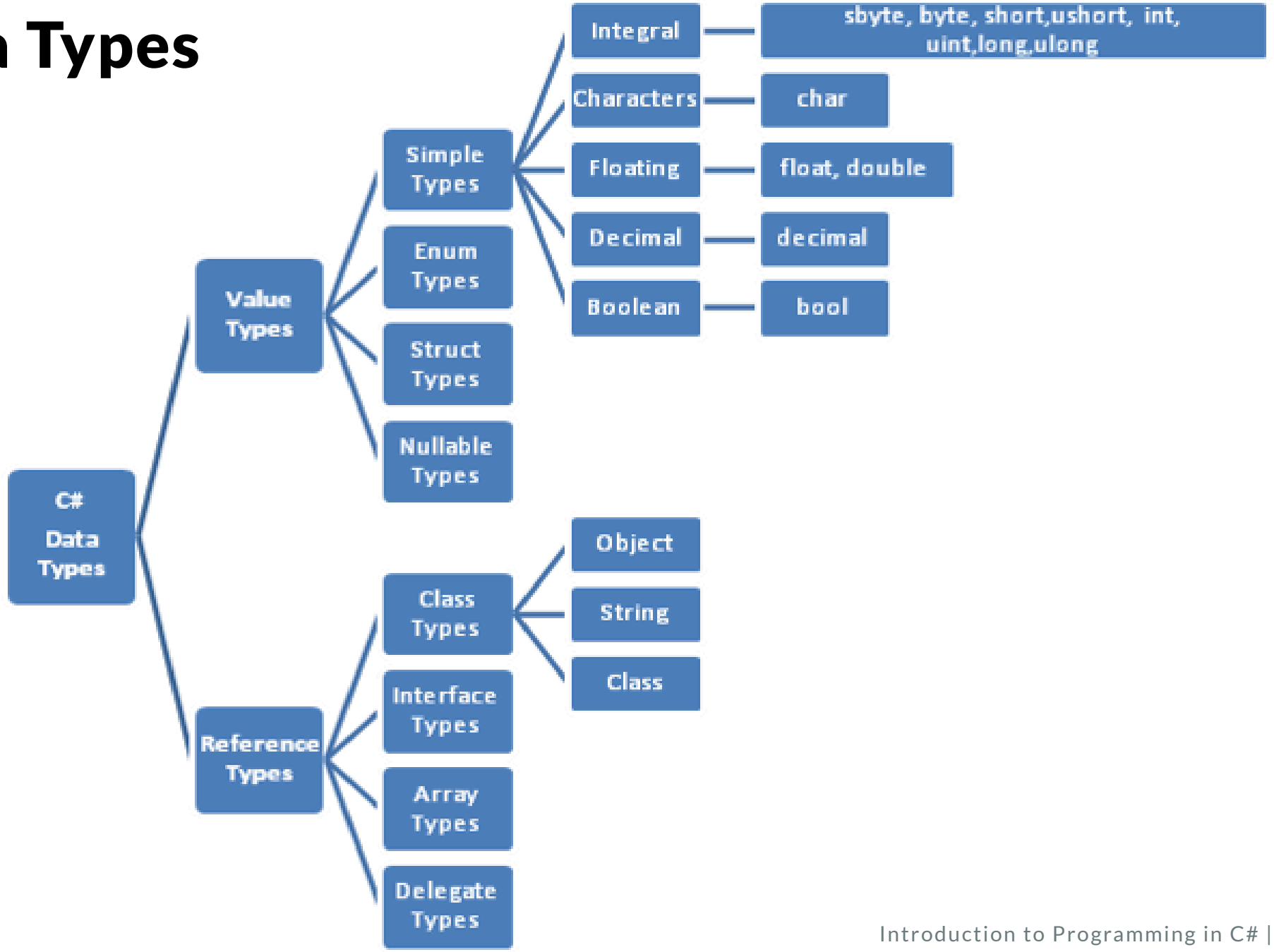
```
// Program.cs
Console.WriteLine("Hello World!");
```

- Comments (/**/, //, ///)
- Function call
- Console I/O:
  Console.WriteLine(...), Console.ReadLine(...)

C# Naming Conventions

Identifyer	Casing	Example
Namespace	Pascal	namespace System.Security { … }
Type	Pascal	public class StreamReader { … }
Interface	Pascal	public interface IEnumerable { … }
Method	Pascal	public virtual string ToString() { … }
Property	Pascal	public int Length { get; }
Event	Pascal	public event EventHandler Exited;
Field	Pascal	public static readonly TimeSpan InfiniteTimeout; public const Min = 0;
Enum value	Pascal	public enum FileMode { Append, … }
Parameter	Camel	public static int ToInt32(string value);

C# Data Types


C# Data Types
Value Types
Simple Types
Integral
sbyte, byte, short,ushort, int, uint,long,ulong
Characters
char
Floating
float, double
Decimal
decimal
Boolean
bool
Enum Types
Struct Types
Nullable Types
Reference Types
Class Types
Object
String
Class
Interface Types
Array Types
Delegate Types

C# Data Types

Consider with data types…

- Stack vs. Heap
- Scopes (Blocks)
- Arguments of functions are handed-in: *by-value* (copy) or *by-reference*
- Variable declaration
  Constants declaration
  Initialization
- Enum: hold a list of named constants grouped inside a „type“.

Expressions

*“In computer science, an expression is a **syntactic entity** in a programming language that may be evaluated to determine its value.

It is a **combination** of one or more constants, variables, functions, and operators that the programming language interprets (according to its particular rules of precedence and of association) and **computes to** produce (“to return”, in a stateful environment) **another value**.

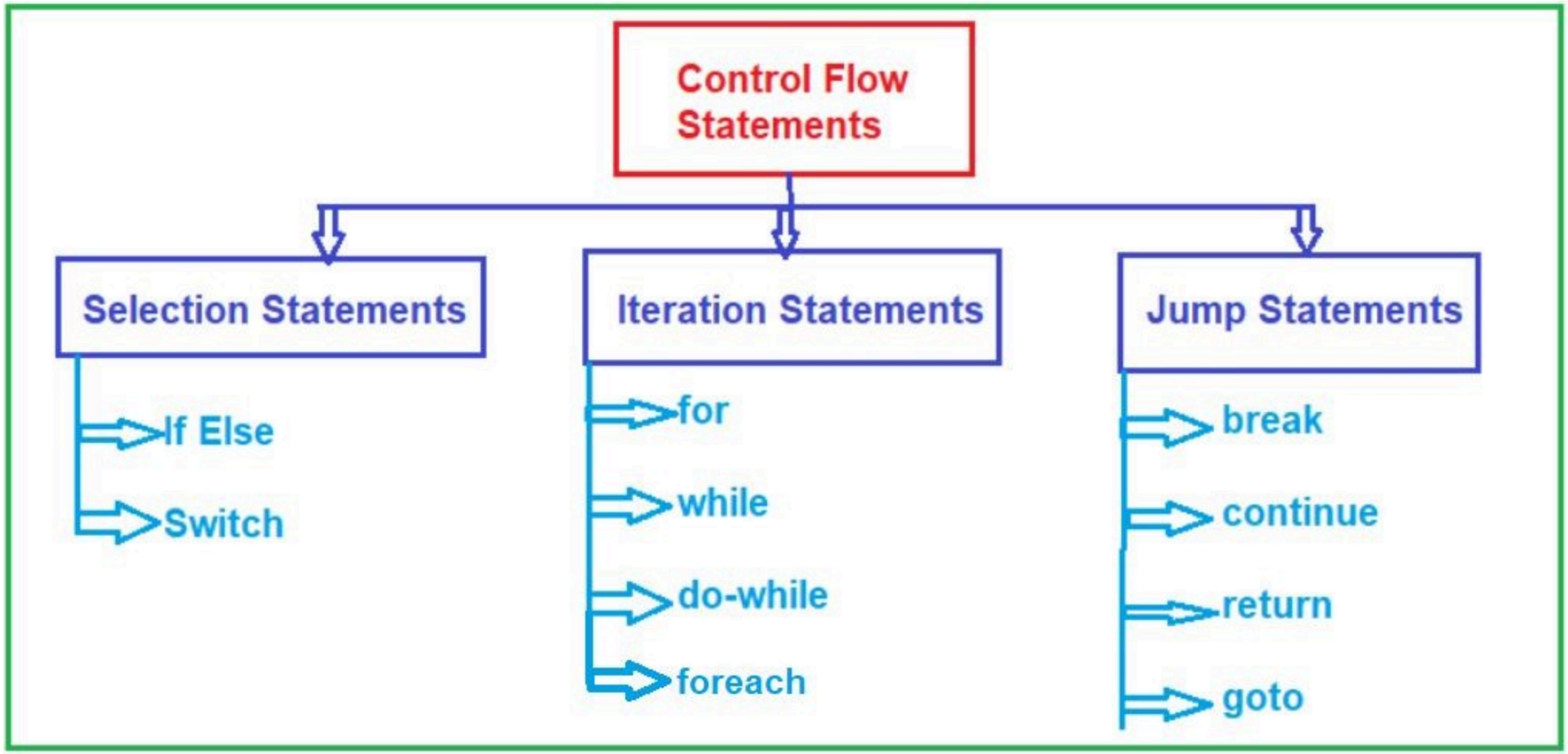
This process, for mathematical expressions, is called evaluation.”*
[Wikipedia]

C# Operators

Operators	Category or name
`x.y, f(x), a[i], x?.y, x?[y], x++, x--, x!, new, typeof, checked, unchecked, default, nameof, delegate, sizeof, stackalloc, x->y`	Primary
`+x, -x, !x, ~x, ++x, --x, ^x, (T)x, await, &x, *x, true, false`	Unary
`x..y`, `switch, with`	Range, then switch and with
`x * y, x / y, x % y`	Multiplicative
`x + y, x – y`	Additive
`x << y, x >> y, x >>> y`	Shift
`x < y, x > y, x <= y, x >= y, is, as`	Relational and type-testing
`x == y, x != y`	Equality
`x & y`, `x ^ y`, `x | y`	Boolean/Bitwise logical AND, then XOR, then OR
`x && y`, `x || y`	Conditional AND, then OR
`x ?? y`, `c ? t : f`	Null-coalescing operator, then Conditional operator
`x = y, x += y, x -= y, x *= y, x /= y, x %= y, x &= y, x |= y, x ^= y, x <<= y, x >>= y, x >>>= y, x ??= y, =>`	Assignment and lambda declaration

see https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/#operator-precedence

C# Flow Control



C# Coalscing Operators - proper handle nullables

- ? ... nullable declaration

```
int? a = null;
```

- ?? Operator ... null-coalescing operator

```
int? a = null;
int b = a ?? -1;
```

- ??= ... null-coalescing assignment operator

```
List<int> numbers = null;
(numbers ??= new List<int>()).Add(5);
```

- ?. and ?[] ... Null-conditional operators
  - If a evaluates to null, the result of a?.x or a?[x] is null.
  - If a evaluates to non-null, the result of a?.x or a?[x] is the same as the result of a.x or a[x], respectively.

```
return setsOfNumbers?[indexOfSetToSum]?.Sum() ?? double.NaN;
```

Further Reading

- A tour of the C# language: https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/

Microsoft Docs:

- .Net: https://docs.microsoft.com/en-us/dotnet/
- Docs: https://docs.microsoft.com/en-us/dotnet/csharp/
- Language reference: https://docs.microsoft.com/en-us/dotnet/csharp/language-reference/
- Channel9 Videos: https://channel9.msdn.com/Series/CSharp-101/What-is-C/
- Easy Tutorial: https://www.tutorialspoint.com/csharp/index.htm